

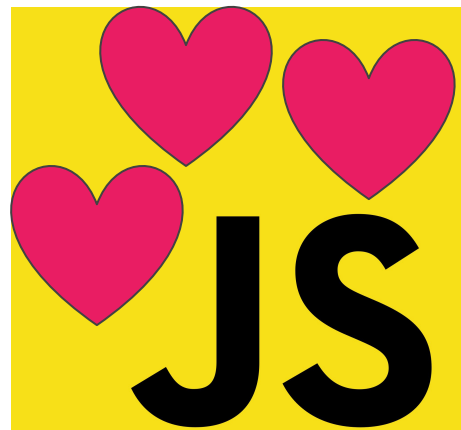
COMP1531

Software Engineering Fundamentals

Week 1
JavaScript

JavaScript

- A very...quirky programming language
 - Designed in 10 days back in 1995 so Netscape could add scripting to their browser
 - Once a feature shipped to the web, it was locked in
 - JavaScript evolved in real-time, on billions of devices
- A very appropriate language for us to learn
 - one of the most in-demand programming languages.
 - Necessary for your lab and project :-)
- [Fun \(OPTIONAL\) Demo](#)



JavaScript

- If it's a mess, why learn it?
 - It's literally the most important programming language in the world
 - Industry Relevance: Major companies like Google, Facebook, and Netflix use JavaScript extensively.
 - 99% of websites use JavaScript in some capacity
 - Versatile
 - Frontend
 - Backend
 - Rapid Prototyping
 - Javascript has an extremely rich open source library and package manager that allows you to build apps quickly. Javascript is very high level, making it easy to write code.



Disclaimer

Because you already know C, we will teach Javascript very quickly and mainly focus on the differences between Javascript and C.

Don't expect to know everything about Javascript this term. Just focus on only learning what you need to solve a problem at hand, and you will learn more super quick.

Javascript

```
1  const z = 3;  
2  function hello(a, b, c) {  
3    return `${a} ${b}`;  
4  }
```

intro.js

JavaScript vs C

— — —

Write a function that takes in two numbers, and returns the smaller number

C

```
1 int minimum(int a, int b) {  
2     if (a > b) {  
3         return b;  
4     } else {  
5         return a;  
6     }  
7 }
```

compare_1.c

JavaScript

```
1 function minimum(a, b) {  
2     if (a > b) {  
3         return b;  
4     } else {  
5         return a;  
6     }  
7 }
```

compare_1.js

JavaScript vs C

— — —

Write a function that takes in two numbers, and returns the smaller number

C

```
1 int minimum(int a, int b) {  
2     if (a > b) {  
3         return b;  
4     } else {  
5         return a;  
6     }  
7 }
```

`compare_1.c`

JavaScript

```
1 function minimum(a, b) {  
2     if (a > b) {  
3         return b;  
4     } else {  
5         return a;  
6     }  
7 }
```

`compare_1.js`

Now let's call the function and print the result!

JavaScript vs C

Write a function that takes in two numbers, and returns the smaller number

— — —

C

```
1 #include <stdio.h>
2
3 int minimum(int a, int b) {
4     if (a > b) {
5         return b;
6     } else {
7         return a;
8     }
9 }
10
11 int main(int argc, char* argv[]) {
12     printf("%d\n", minimum(3, 5));
13 }
```

compare_2.c

```
1 gcc -Wall -Werror -O -o compare_2.c -o runnable
2 ./runnable
```

JavaScript

```
1 function minimum(a, b) {
2     if (a > b) {
3         return b;
4     } else {
5         return a;
6     }
7 }
8
9 console.log(minimum(3, 5));
```

compare_2.js

```
1 node compare_2.js
```


What is NodeJS?

- Compiles and runs JavaScript
 - Allows JS to be run outside of the browser
- Similar to GCC (functionality wise)



What is NodeJS?

- NodeJS is an interpreted language
 - Compiled and run at the same step
- A little slower (it has to compile to runnable code every time.
- A little more convenient (changes to code don't require an extra compilation step)

Performance vs Convenience



JavaScript vs C

- Let's learn some more fun JavaScript!

The JavaScript logo, consisting of the letters 'JS' in a bold, black, sans-serif font, centered within a solid yellow square.

JS

Variables and Printing

Variables are containers for data values. `const`, `let`, `console.log`

— — —

```
1 // Variables declared with "let"
2 // can be modified after definition
3 let years = 5;
4
5 // Variables declared with "const"
6 // cannot be modified after definition
7 const fullname = "Giraffe";
8 const age = 18;
9 const height = 2048.11;
10
11 // // You print with console.log
12 console.log(years);
13
14 // // Double and single apostrophes are equivalent
15 console.log('Hello!');
16 console.log("how are you?");
```

`variables.js`

Strings

Concatenation, string literals

— — —

```
1 // // We can easily join strings together!
2 let sentence = 'My';
3 sentence = sentence + " nick" + ' name is';
4 sentence += ' Pikachu';
5 console.log(sentence);
6
7 // If you need to mix variables and
8 // strings, you can create a string literal
9 const age = 7;
10 const fullname = 'Yuchao';
11 const phrase = `Hello! My name is ${fullname} and I am ${age}`;
12 console.log(phrase);
```

[strings.js](#)

Literals provide an easy way to interpolate variables and expressions into strings. The method is called string interpolation (using backtick `)

Control Structures

if, else if, else, while, for.

— — —

```
1  const number = 5;
2  if (number > 10) {
3    console.log('Bigger than 10');
4  } else if (number < 2) {
5    // Do nothing
6  } else {
7    console.log('Number between 2 and 9');
8  }
9
10 console.log('-----');
11
12 let i = 0;
13 while (i < 5) {
14   console.log('Hello there');
15   i += 1;
16 }
17
18 console.log('-----');
19
20 for (let i = 0; i < 5; i++) {
21   console.log('Hello there');
22 }
```

Functions

Very similar syntax to C.

— — —

```
1 function minimum(a, b) {  
2   if (a > b) {  
3     return b;  
4   } else {  
5     return a;  
6   }  
7 }
```

compare_1.js

Data Structures: Collections

We'll now discuss two important data structures that are both collections of data.

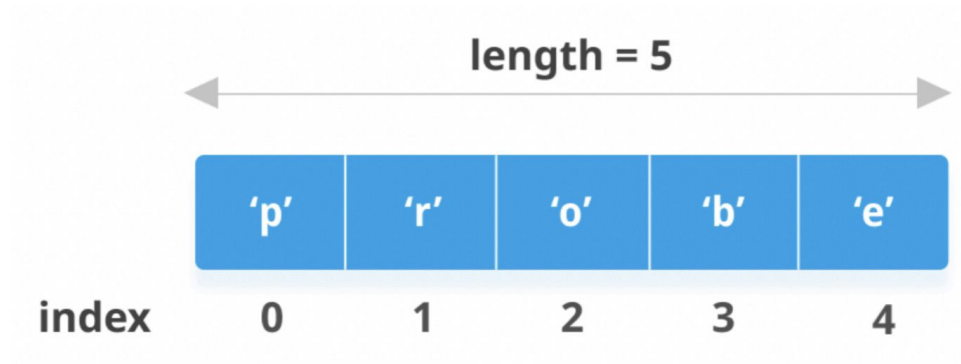
Collections can either be:

- Sequential collections
- Associative collections

Sequential Collections

In sequential collections values are referenced by their integer index (key) that represents their location in an order.

In Javascript sequential collections are represented by an array.



Associative Collections

In associative collections values are referenced by their string key that maps to a value.

They often do not have an inherent sense of order.

- **name** → "Shaveen"
- **age** → 18
- **height** → "178cm"

Arrays

An array is a special variable. An array can hold many values under a single name, and you can access the values by referring to an index number. Arrays are dynamic and can hold different data types.

```
1 // This is a array
2 const names = ['Yuchao', 1, 1.2];
3
4 // Dynamic arrays that can hold different data types
5 names.push('Giuliana');
6 names.unshift('Rani');
7 names[1] = 'Carl';
8 console.log(`Hi ${names}`);
```

Arrays

Iterate through an array.

```
1 const items = ['a', 'b', 'c', 'd', 'e'];
2
3 let i = 0;
4 while (i < 5) {
5   console.log(items[i]);
6   i++;
7 }
8
9 for (let j = 0; j < 5; j++) {
10  console.log(items[j]);
11 }
12
13 for (let k = 0; k < items.length; k++) {
14  console.log(items[k]);
15 }
```

[array_basic.js](#)

Array

For In Loop v.s. For Of Loop

```
1 const items = ['a', 'b', 'c', 'd', 'e'];
2
3 // prints 0, 1, 2, 3, 4
4 for (const i in items) {
5   console.log(i);
6 }
7
8 // prints a, b, c, d, e
9 for (let item of items) {
10  console.log(item);
11 }
```

[array_advanced.js](#)

- For In statement loops through the keys/indexes of an array.
- For Of statement loops through the values of an array.

Objects

[Source](#)

Real Life Objects, Properties, and Methods

In real life, a car is an **object**.

A car has **properties** like weight and color, and **methods** like start and stop:

Object	Properties	Methods
	<code>car.name = Fiat</code> <code>car.model = 500</code> <code>car.weight = 850kg</code> <code>car.color = white</code>	<code>car.start()</code> <code>car.drive()</code> <code>car.brake()</code> <code>car.stop()</code>

All cars have the same **properties**, but the property **values** differ from car to car.

All cars have the same **methods**, but the methods are performed **at different times**.

Objects

- Objects are variables too. But objects can contain many values.
- This code assigns many values (Fiat, 500, white) to a variable named car:

```
const object = {make: "Toyota", model: "Yarris", year: 2014, colour: "blue"};
```

- The values are written as name:value pairs (name and value separated by a colon).
- It is a common practice to declare objects with the const keyword.

Objects

- Objects are associative structures that may consist of many different types.
- You can use them when you need a collection of items that are identified by a string description, rather than a numerical index (arrays).

```
1 const student = {  
2   name: 'Rani',  
3   score: 99,  
4   rank: 1,  
5 };  
6  
7 student.score = 100;  
8 student['zID'] = 'z123456';  
9  
10 console.log(student);  
11 console.log(student.score);
```


Objects

- We can create and populate objects in different ways.

```
1 const userData = {};  
2 userData.name = 'Sally';  
3 userData.age = 18;  
4 userData.height = '187cm';  
5 console.log(userData);
```

[object_basic1.js](#)

```
1 const userData = {  
2   name: 'Sally',  
3   age: 18,  
4   height: '187cm',  
5 };  
6 console.log(userData);
```

[object_basic2.js](#)

Both of these programs would print: { name:'Sally', age: 18, height:'187cm'}

Objects

We can mix the two methods, and also use alternative syntax with assigning.

```
1 userData.prop = 1;  
2 userData['prop'] = 1;
```

```
1 // You can assign more keys even  
2 // after creation  
3 const userData = {  
4   name: 'Sally',  
5   age: 18,  
6 };  
7 userData.height = '187cm';  
8  
9 console.log(userData);
```

object_more1.js

```
1 // You can reference keys with either  
2 // obj.key or obj['key']  
3 const userData = {};  
4 userData.name = 'Sally';  
5 userData.age = 18;  
6 userData['height'] = '187cm';  
7 console.log(userData);
```

object_more2.js

Objects

We can also get various properties of an object using the Object functions

```
1  const userData = {  
2    name: 'Sally',  
3    age: 18,  
4    height: '187cm',  
5  };  
6  
7  const keys = Object.keys(userData);  
8  const entries = Object.entries(userData);  
9  const values = Object.values(userData);  
10  
11 console.log(keys);  
12 console.log(entries);  
13 console.log(values);
```

object_props_1.js

Objects

The output:

```
[ 'name', 'age', 'height' ]  
[ [ 'name', 'Sally' ], [ 'age', 18 ], [ 'height', '187cm' ] ]  
[ 'Sally', 18, '187cm' ]
```

Further Discussion Of Object

- The following code exhibits behaviour you're probably not used to:

```
1 const arr = [1, 2, 3];  
2 console.log(arr.length);  
3 console.log(arr.includes(3));
```

[object_model.js](#)

- "arr" is an array, but it also seems to have:
 - A property length that we never set?
 - Some kind of function that is being called?
 - Let's look at why this is.

Arrays are Objects !!

- An “object” is a data type which
 - Contains 0 or more properties/attributes
 - Contains 0 or more methods/functions
- Arrays fit this criteria??
 - So do strings??
 - So do functions???

Tying Some Things Together

Let's try some lists of objects (an array of objects).

object_loop1.js

```
1  const userData = [  
2    {  
3      name: 'Sally',  
4      age: 18,  
5      height: '186cm',  
6    }, {  
7      name: 'Bob',  
8      age: 17,  
9      height: '188cm',  
10   },  
11  ];  
12  
13  // Returns an array of the object's keys  
14  const keys = Object.keys(userData);  
15  console.log(keys);  
16  
17  // Returns an array of the object's  
18  // [key, value] pairs.  
19  const entries = Object.entries(userData);  
20  console.log(entries);  
21  
22  // Returns an array of the object's values.  
23  const values = Object.values(userData);  
24  console.log(values);
```

Tying Some Things Together

Let's try some lists of objects (an array of objects).

```
1  const userData = [  
2    {  
3      name: 'Sally',  
4      age: 18,  
5      height: '186cm',  
6    }, {  
7      name: 'Bob',  
8      age: 17,  
9      height: '188cm',  
10   },  
11  ];  
12  
13  for (let i = 0; i < userData.length; i++) {  
14    console.log(`${userData[i].name}'s properties are:`);  
15    console.log(`  name: ${userData[i].name}`);  
16    console.log(`  age: ${userData[i].age}`);  
17    console.log(`  height: ${userData[i].height}`);  
18  }
```

object_loop2.js

Tying Some Things Together

Let's try some lists of objects.

```
1  const userData = {
2    Sally: {
3      age: 18,
4      height: '186cm',
5    },
6    Bob: {
7      age: 17,
8      height: '188cm',
9    },
10 };
11
12 for (const key in userData) {
13   console.log(`${key}'s properties are:`);
14   for (const key2 in userData[key]) {
15     console.log(`  ${key2}: ${userData[key][key2]}`);
16   }
17 }
```

object_loop3.js

Tying Some Things Together

Let's try more lists of objects.

```
1 const student1 = { name: 'Yuchao', score: 50 };
2 const student2 = { name: 'Rani', score: 91 };
3 const student3 = { name: 'Giuliana', score: 99 };
4 const students = [student1, student2, student3];
5
6 console.log(students);
7
8 // Approach 1
9 const numStudents = students.length;
10 for (let i = 0; i < numStudents; i++) {
11   if (students[i].score >= 85) {
12     console.log(`${students[i].name} got an HD`);
13   }
14 }
15
16 // Approach 2
17 for (const student of students) {
18   if (student.score >= 85) {
19     console.log(`${student.name} got an HD`);
20   }
21 }
```

combining.js

Further Reading

— — —

- [Array of Objects](#)

Woah!

- This may seem very fast and confusing
 - So how can you practice?



Practice Available

— — —

- Labs
- Help Sessions
- Extra Resources!
 - New optional material developed for practising programming this term
 - If you find them useful, let us know!
 - We might make more if demand is high enough

Feedback

— — —



Or go to the [form here](#)